

รายงานสรุปผล

Socket Programming

Bid Auction Protocol (BAP/1.0)

หลักการการสื่อสารคอมพิวเตอร์และการประมวลผลบนคลาวด์

Computer Communications and Cloud Computing Principles

รหัสวิชา 01418351

จัดทำโดย

ป้อมเดช ฟุง

6710451046

ภาควิชาวิทยาการคอมพิวเตอร์ มหาวิทยาลัยเกษตรศาสตร์

1. บทนำและวัตถุประสงค์

1.1 วัตถุประสงค์ของโปรแกรม

โปรแกรมนี้คือระบบประมูลแบบกระจาย (Distributed Bid Auction System) ที่ให้ผู้ใช้หลายคน สามารถเสนอราคา (bid) สำหรับสินค้าประมูลที่มีหมายเลขกำกับ (item ID ตั้งแต่ 1 ถึง 200) ผ่านเครือข่าย โดยระบบจะรับประกันว่า ณ เวลาใดเวลาหนึ่ง จะมีผู้เสนอราคาสูงสุดเพียงคนเดียวต่อสินค้าหนึ่งชิ้นเสมอ หากมีผู้เสนอราคาต่ำกว่าราคาสูงสุดปัจจุบัน ระบบจะปฏิเสธการเสนอราคาร้านั้นทันที

จุดประสงค์ของการออกแบบระบบในลักษณะนี้ คือการสาธิตสถาปัตยกรรมแบบ client-server ที่มีการแบ่ง ภาระงานตาม ช่วงของ item ID (sharding) โดยแต่ละช่วงจะถูกดูแลโดยคู่ของ proxy และ server เฉพาะของตนเอง และมี registry ทำหน้าที่เป็นตัวกลางให้ client และ proxy ค้นหาได้แบบพลวัต (dynamic) แทนที่จะต้อง กำหนด address กันไว้ล่วงหน้า แบบตายตัวในโค้ด

1.2 คุณลักษณะของแอปพลิเคชัน

- สถาปัตยกรรมแบบ multi-tier client-server มี 4 บทบาทที่แยกจากกันชัดเจน คือ Client, Proxy, Server และ Registry
- แบ่งช่วงของ item ID ออกเป็นส่วนที่ไม่ทับซ้อนกัน (sharding) เช่น 1-100 และ 101-200 โดยแต่ละช่วงมี Proxy และ Server คู่ของตนเอง
- Proxy ไม่มีสถานะ (stateless) ทำหน้าที่เพียงตรวจสอบช่วง item ID และส่งต่อคำขอ ส่วน Server มีสถานะ (stateful) เป็นผู้ถือครองราคาสูงสุดของแต่ละสินค้าในช่วงที่ตนดูแลมีกลไกค้นหาแบบพลวัต (dynamic discovery) โดย Proxy จะลงทะเบียนช่วงและ address ของตนกับ Registry ตอนเริ่มทำงาน และ Client จะสอบถาม Registry ทุกครั้งก่อนส่ง bid
- รูปแบบการสื่อสารเป็นแบบ request-response หนึ่งคำขอต่อหนึ่งคำตอบ ไม่ใช้การสตรีมข้อมูลต่อเนื่อง
- มีการบันทึกข้อมูลถาวรลงดิสก์ (persistence) ทั้ง audit log และ snapshot เพื่อให้ระบบกู้คืนสถานะได้หากโปรเซสหยุดทำงานกะทันหัน

1.3 เหตุผลในการเลือกใช้ Transport Layer แบบ TCP

ระบบนี้เลือกใช้ TCP (ผ่านแพ็คเกจ net ของภาษา Go โดยใช้ net.Listen("tcp", ...) และ net.Dial("tcp", ...)) แทนที่จะใช้ UDP ด้วยเหตุผลดังต่อไปนี้

1.3.1. **ความน่าเชื่อถือเป็นสิ่งจำเป็น ไม่ใช่ทางเลือก:** การเสนอราคาถือเป็นธุรกรรมที่มีความสำคัญเชิงความหมาย หากข้อความ bid สูญหายระหว่างทาง (ซึ่ง UDP อนุญาตให้เกิดขึ้นได้) ผู้ใช้จะเข้าใจผิดว่าตนได้เสนอราคาไปแล้วทั้งที่ server ไม่เคยได้รับเลย

1.3.2 **ลำดับของข้อมูลมีผลกระทบต่อผลลัพธ์:** การที่ bid ของ Bob จำนวน 150 บาทถูกประมวลผลก่อนหรือหลัง bid ของ Carol จำนวน 120 บาท จะกำหนดผลของการประมูลในสินค้านั้นโดยตรง TCP รับประกันการส่งข้อมูลตามลำดับในหนึ่ง connection ในขณะที่ UDP ไม่รับประกันเรื่องนี้เลย

1.3.3. **การเชื่อมต่อแบบมี connection ช่วยให้แยกแยะข้อผิดพลาดได้ชัดเจน:** BAP/1.0 จำเป็นต้องแยกความแตกต่างระหว่าง "ปลายทางติดต่อไม่ได้" กับ "ปลายทางตอบกลับด้วยสถานะผิดพลาด" การทำ 3-way handshake ของ TCP ทำให้ทราบทันทีว่าปลายทางไม่ตอบสนอง ซึ่ง Proxy นำไปใช้ตอบกลับ client ด้วยสถานะ 503 Service Unavailable ได้ อย่างชัดเจน

1.3.4. **ขนาดข้อความเล็กและความถี่ต่ำ:** แต่ละ bid เป็น JSON object เพียงชิ้นเดียว ทำให้ overhead ของ TCP ที่มากกว่า UDP เล็กน้อยไม่ใช่ต้นทุนที่มีนัยสำคัญ เมื่อเทียบกับความถูกต้องของข้อมูลที่ได้มา

2. การออกแบบ Application-Layer Protocol: BAP/1.0

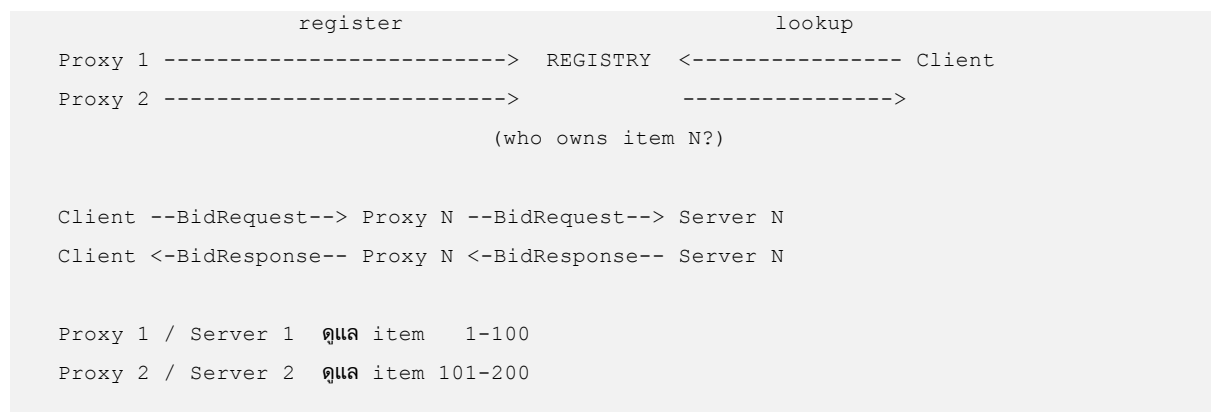
2.1 ชื่อโปรโตคอลและรูปแบบข้อมูลบนสาย (Wire Format)

โปรโตคอลระดับ application ที่ออกแบบขึ้นสำหรับระบบนี้มีชื่อว่า Bid Auction Protocol เรียกโดยย่อว่า BAP รุ่นปัจจุบันคือ 1.0 ระบุบนสายสื่อสารด้วยข้อความ "BAP/1.0" ซึ่งบรรจุอยู่ในฟิลด์ protocol ของทุกข้อความ ในลักษณะเดียวกับที่ HTTP request line ระบุ "HTTP/1.1"

รูปแบบข้อมูลบนสาย: ทุกข้อความของ BAP/1.0 เป็น JSON object หนึ่งชิ้น ใช้รหัสแบบ UTF-8 และปิดท้ายด้วยอักขระขึ้นบรรทัดใหม่ (\n) เนื่องจากเนื้อหา JSON ของโปรโตคอลนี้ไม่มีอักขระขึ้นบรรทัดใหม่ปะปนอยู่ การแบ่ง byte stream ของ TCP ด้วยอักขระ \n จึงเพียงพอต่อการแยกขอบเขตของแต่ละข้อความ โปรโตคอลนี้เป็นแบบ request-response กล่าวคือทุกข้อความที่ Client หรือ Proxy ส่งออกไป จะต้องได้รับการตอบกลับหนึ่งข้อความบน connection เดียวกันก่อนจะส่งข้อความถัดไป

BAP/1.0 แบ่งกลุ่มข้อความออกเป็น 2 กลุ่มที่เป็นอิสระจากกัน ใช้รูปแบบการวางเฟรมและสแตตัส status code ร่วมกัน คือกลุ่มข้อความ Bid (Client-Proxy-Server) และ กลุ่มข้อความ Registry (Proxy/Client-Registry)

2.2 ภาพรวมสถาปัตยกรรม



2.3 กลุ่มข้อความ Bid (Client ↔ Proxy ↔ Server)

BidRequest — ส่งจาก Client ไปยัง Proxy และถูกส่งต่อโดยไม่มีการแก้ไขไปยัง Server ที่จับคู่กัน

Field	Type	Description
protocol	string	ตัวระบุโปรโตคอล ค่าคงที่คือ "BAP/1.0"
client_name	string	ชื่อที่แสดงของผู้เสนอราคา
item_id	int	หมายเลขสินค้าที่เสนอราคา (1-200)
bid_amount	float	จำนวนเงินที่เสนอราคา

BidResponse — ส่งจาก Server กลับไปยัง Proxy และถูกส่งต่อไปยัง Client

Field	Type	Description
protocol	string	ตัวระบุโปรโตคอล ค่าคงที่คือ "BAP/1.0"
status_code	int	รหัสผลลัพธ์เชิงตัวเลข (ดูตารางในหัวข้อ 2.3.1)
status_phrase	string	ข้อความอธิบายที่คู่กับ status_code
reason	string	(ไม่บังคับ) รายละเอียดเพิ่มเติม เช่น เหตุผลที่ถูกปฏิเสธ
item_id	int	(ไม่บังคับ) สินค้าที่เกี่ยวข้องกับคำตอบนี้
highest_bid	float	(ไม่บังคับ) ราคาสูงสุดปัจจุบันของสินค้านั้น
highest_bidder	string	(ไม่บังคับ) ชื่อผู้เสนอราคาสูงสุดปัจจุบัน

รหัสสถานะของกลุ่มข้อความ Bid

Status	Message	Meaning
200	OK - Bid Accepted	การเสนอราคานี้กลายเป็นราคาสูงสุดของสินค้าชิ้นนี้แล้ว
409	Conflict - Bid Too Low	มีการเสนอราคาที่สูงกว่านี้อยู่ก่อนแล้วสำหรับสินค้าชิ้นนี้
404	Not Found - Item Out Of Range	Proxy ที่ติดต่อไปไม่ได้ดูแลช่วง item ID นี้
503	Service Unavailable - Server Unreachable	ไม่สามารถติดต่อ Server ที่จับคู่กับ Proxy นี้ได้

2.4 กลุ่มข้อความ Registry (Proxy/Client ↔ Registry)

Registry คือสิ่งที่ทำให้ Proxy เป็นแบบพลวัต (dynamic) แทนที่จะใช้ตารางที่กำหนดตายตัวไว้ในโค้ด Proxy แต่ละตัวจะประกาศตัวเองกับ Registry ตอนเริ่มทำงาน และ Client จะสอบถาม Registry ทุกครั้งก่อนส่ง bid ว่า item ID นั้นควรติดต่อ Proxy ตัวใด ข้อความประเภทเดียวคือ RegistryMessage ถูกใช้ทั้งสองปฏิบัติการ โดยแยกแยะด้วยฟิลด์ type

Field	Type	Description
protocol	string	ตัวระบุโปรโตคอล ค่าคงที่คือ "BAP/1.0"
type	string	"register" (ส่งโดย Proxy) หรือ "lookup" (ส่งโดย Client)
proxy_id	int	(register) หมายเลขเฉพาะของ Proxy ที่ลงทะเบียน
low / high	int	(register) ช่วง item ID ที่ Proxy นี้ดูแล
proxy_host / proxy_port	string	(register) address ที่ Proxy นี้เปิดรับการเชื่อมต่อ
item_id	int	(lookup) หมายเลขสินค้าที่ Client ต้องการเสนอราคา

2.4.1 รหัสสถานะของกลุ่มข้อความ Registry

Status	Message	Meaning
200	OK	ลงทะเบียนสำเร็จ หรือค้นหาแล้วพบ Proxy ที่ตรงกัน
404	Not Found - No Proxy Registered For Item	ไม่มี Proxy ที่ลงทะเบียนดูแล item ID นี้ในขณะนี้
400	Bad Request - Unknown Message Type	ฟิลด์ type ไม่ใช่ทั้ง "register" และ "lookup"

2.5 ตัวอย่างลำดับข้อความ (Message Flow)

ตัวอย่างต่อไปนี้แสดงลำดับข้อความทั้งหมดของ BAP/1.0 เมื่อมีผู้เสนอราคาสองคนแข่งกันบนสินค้าชิ้นเดียวกัน (item 42 ซึ่งอยู่ในความดูแลของ Proxy 1 / Server 1)

โดย Alice เสนอราคาก่อนและได้รับการยอมรับ จากนั้น Bob เสนอราคาสูงกว่าและได้รับการยอมรับเช่นกัน สุดท้าย Carol พยายามเสนอราคาต่ำกว่า Bob จึงถูกปฏิเสธ

```
1. Alice's client -> Registry:  lookup item_id=42

   Registry -> Alice's client:  200 OK  (proxy_id=1, 127.0.0.1:7001)

2. Alice's client -> Proxy 1:  BidRequest{client_name=Alice, item_id=42, bid_amount=100}

   Proxy 1 -> Server 1:          BidRequest{client_name=Alice, item_id=42,
bid_amount=100}

   Server 1 -> Proxy 1:          200 OK - Bid Accepted (highest_bid=100,
highest_bidder=Alice)

   Proxy 1 -> Alice's client:    200 OK - Bid Accepted (highest_bid=100,
highest_bidder=Alice)

3. Bob's client bids 150 บนสินค้าเดียวกัน -> 200 OK - Bid Accepted

4. Carol's client -> Proxy 1:    BidRequest{client_name=Carol, item_id=42,
bid_amount=120}

   Proxy 1 -> Server 1:          BidRequest{client_name=Carol, item_id=42,
bid_amount=120}

   Server 1 -> Proxy 1:          409 Conflict - Bid Too Low (highest_bid=150,
highest_bidder=Bob)

   Proxy 1 -> Carol's client:    409 Conflict - Bid Too Low (highest_bid=150,
highest_bidder=Bob)
```

3. สถาปัตยกรรมของระบบและกลไกการทำงาน

3.1 บทบาทของแต่ละองค์ประกอบ

- Client — ส่งคำขอเสนอราคา โดยไม่จำเป็นต้องรู้ล่วงหน้าว่า Proxy หรือ Server ตัวใดดูแลสินค้าที่ต้องการ
- Registry — ทำหน้าที่เป็นสมุดที่อยู่แบบพลวัต เก็บว่า Proxy ตัวใดดูแล item ID ช่วงใด และตอบคำถามการค้นทำให้ Client
- Proxy — ตรวจสอบว่า item ID ที่ขอเข้ามาอยู่ในช่วงที่ตนดูแลหรือไม่ หากใช่จะส่งต่อไปยัง Server ที่จับคู่กัน หากไม่ใช่จะตอบกลับด้วยรหัส 404 ทันทีโดยไม่ต้องติดต่อ Server เลย
- Server — เป็นเจ้าของสถานะที่แท้จริงเพียงหนึ่งเดียวสำหรับช่วง item ID ของตน เก็บราคาสูงสุดปัจจุบันของแต่ละสินค้า และเป็นผู้ตัดสินใจว่าจะรับหรือปฏิเสธการเสนอราคา

3.2 การลงทะเบียน Proxy แบบพลวัต (Dynamic Proxy Registration)

แทนที่จะกำหนด address ของ Proxy แต่ละตัวไว้ตายตัวในโค้ดที่ใช้ร่วมกันทุกโปรเซส Proxy แต่ละตัวจะถูกส่งรัน พร้อมค่าพารามิเตอร์ของตนเอง (หมายเลข ID, ช่วง item ID, พอร์ตที่จะเปิดรับ, address ของ Server ที่จับคู่กัน) แล้วเชื่อมต่อไปยัง Registry เพื่อประกาศข้อมูลของตนเองตอนเริ่มทำงาน ส่วน Client จะสอบถาม Registry ทุกครั้ง ก่อนส่ง bid ว่าควรติดต่อ Proxy ไດ ผลลัพธ์คือสามารถเพิ่ม shard ใหม่ หรือเพิ่ม instance ของ Proxy เดิมเพื่อรองรับ โหลดที่มากขึ้น ได้โดยไม่ต้องแก้ไขโค้ดของ Client, Proxy หรือ Server ตัวอื่นเลย

3.3 กลไกด้าน Data Integrity

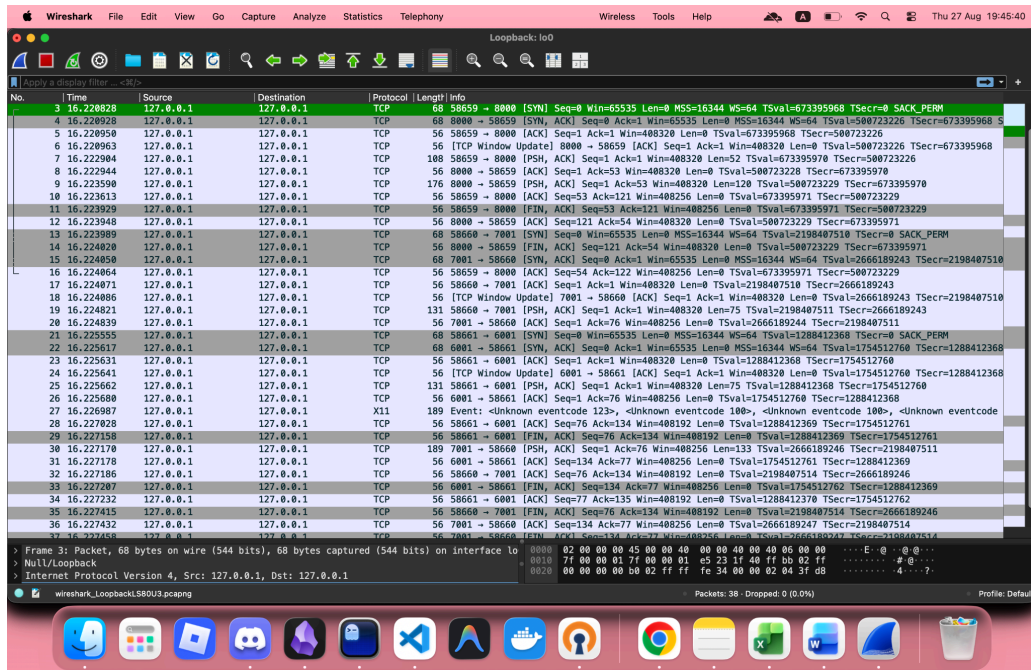
TCP รับประกันเพียงว่าข้อมูลที่ส่งออกไปจะถึงปลายทางอย่างถูกต้องและเรียงลำดับ แต่ไม่รับประกันเรื่องที่เกิดขึ้น หลังจากข้อมูลถึงปลายทางแล้ว เช่น การแข่งขันกันเขียนข้อมูล (race condition) การสูญหายของข้อมูลเมื่อโปรเซสหยุดทำงานกะทันหัน (crash) หรือการมีโปรเซสซ้ำซ้อนดูแล shard เดียวกัน ระบบนี้จึงเพิ่มกลไกต่อไปนี้ในระดับ application

- Mutex — ป้องกันมิให้มีการเสนอราคาสองรายการเข้ามาพร้อมกันแก้ไขค่าราคาสูงสุดของสินค้าขึ้นเดียวกันพร้อมกัน โดยบังคับให้การอ่าน-ตัดสินใจ-เขียนทั้งหมดเกิดขึ้นทีละรายการ
- Audit log — บันทึกทุกการเสนอราคาที่เข้ามา ไม่ว่าจะได้รับการยอมรับหรือถูกปฏิเสธ ลงในไฟล์ .txt แบบ append-only เพื่อเป็นหลักฐานย้อนหลัง
- Snapshot — บันทึกสถานะปัจจุบัน (ราคาสูงสุดของทุกสินค้าในช่วงนั้น) ลงไฟล์ .txt ทุกครั้งที่มีการยอมรับ bid ใหม่ และโหลดกลับขึ้นมาอัตโนมัติเมื่อ Server เริ่มทำงานใหม่ ทำให้ระบบกู้คืนสถานะได้แม้ Server จะหยุดทำงานกะทันหัน

- Lock file (PID-based) — ป้องกันมิให้มี Server สองโพรเซสดูแล shard เดียวกันพร้อมกันโดยไม่ตั้งใจ หากพบว่า มี process เดิมทำงานอยู่จริงจะปฏิเสธการเริ่มทำงานทันที และหาก process เดิม crash ไปแล้วจะตรวจพบและเข้าครอบครอง lock เดิมโดยอัตโนมัติ

4. การทดสอบด้วย Wireshark

เพื่อยืนยันว่าระบบสื่อสารกันจริงผ่าน TCP ตามที่ออกแบบไว้ ได้ทำการดักจับแพ็กเก็ตบน loopback interface (lo0) ขณะรันโปรแกรมจริง ผลลัพธ์แสดงในภาพที่ 1



ภาพที่ 1: ผลการดักจับแพ็กเก็ตด้วย Wireshark บน loopback interface

4.1 สิ่งสังเกตได้จากการดักจับแพ็กเก็ต

- พบ TCP 3-way handshake (SYN → SYN,ACK → ACK) ครบถ้วนสำหรับทุก connection ที่เกิดขึ้น ยืนยันว่าเป็นการสื่อสารผ่าน TCP จริง
- สำหรับ bid หนึ่งครั้ง เกิด TCP connection แยกจากกันถึง 3 เส้นทาง คือ Client-Registry (พอร์ต 8000), Client-Proxy (พอร์ต 7001) และ Proxy-Server (พอร์ต 6001) แต่ละเส้นทางมี handshake และการปิด connection (FIN,ACK) เป็นของตนเอง
- ทุก connection เป็นแบบอายุสั้น (short-lived) คือเปิด ส่ง-รับข้อมูลหนึ่งรอบ แล้วปิดทันที สอดคล้องกับการออกแบบที่ไม่เก็บ connection ไว้ใช้ซ้ำ
- payload ที่ถูกส่ง (เช่น ขนาด 75 ไบต์ของ BidRequest หรือ 133 ไบต์ของ BidResponse) เป็นข้อความธรรมดาที่ไม่เข้ารหัส เมื่อเปิดดูด้วย Follow TCP Stream จะเห็นเนื้อหา JSON ที่มีฟิลด์ "protocol": "BAP/1.0" ปรากฏอยู่จริง ยืนยันว่าโปรโตคอลระดับ application ที่ออกแบบไว้ถูกส่งผ่าน TCP จริงตามที่ระบุในเอกสารฉบับนี้

- พบแพ็กเก็ตประเภท TCP Window Update ปะปนอยู่บ้าง ซึ่งเป็นกลไก flow control ของ TCP ที่ระบบปฏิบัติการจัดการให้โดยอัตโนมัติ ไม่ใช่ส่วนหนึ่งของโปรโตคอลระดับ application ที่ออกแบบขึ้น และไม่มีส่วนใดในโค้ดของโปรแกรมที่สร้างหรือจัดการแพ็กเก็ตประเภทนี้โดยตรง

4.2 สรุปการแบ่ง Layer

จากผลการทดสอบสามารถสรุปการแบ่งชั้นได้อย่างชัดเจนดังนี้ Application layer คือ BAP/1.0 ซึ่งเป็นโปรโตคอลที่ออกแบบเองทั้งหมด กำหนดรูปแบบข้อความ, status code และความหมายของข้อมูล ส่วน Transport layer คือ TCP ซึ่งทำหน้าที่รับประกันการส่งข้อมูลให้ถึงปลายทางอย่างถูกต้องและเรียงลำดับเท่านั้น ระบบนี้จึงไม่ใช่ WebSocket เนื่องจากไม่มีขั้นตอน HTTP Upgrade handshake และไม่มีการใช้ WebSocket frame format ตามมาตรฐาน RFC 6455 แต่อย่างใด การสื่อสารทั้งหมด เป็นการส่ง JSON ที่คั่นด้วยอักขระขึ้นบรรทัดใหม่ ผ่าน raw TCP socket โดยตรง